

Ecole Nationale Supérieure de Génie Mathéatiques et Modélisation

Reconnainssance vocale de langue FON

Les Chaînes de Markov cachées

Dr Bruno DANDOGBESSI

Email : bdandogbessi@gamil.com

Groupe 1

Membres du groupe

- ANATO Krisland
- AVADRA Martial
- BALOGOUN Charmant
- DOSSOU YOVO Thérèse
- GNONLONFOUN Octave
- OUSMANE Aymane

Table des matières

1. Introduction
2. Notion de Chaîne de Markov Cachée
3. Description du Corpus Utilisé
4. Étapes de Traitement
5. Modèle speechbrain/asr-wav2vec2-dvoice-fongbe
6. Résultats
7. Interface d'Utilisation
8. Avantages, Limites et Améliorations
9. Conclusion
10. Bibliographie
11. Annexes

Introduction

La **reconnaissance vocale automatique (ASR)** est un domaine clé du traitement automatique du langage naturel (NLP), permettant de convertir la parole en texte. Ce projet, intitulé **GbeCé**, se concentre sur la reconnaissance vocale de la langue **fon**, une langue tonale parlée principalement au Bénin et dans certaines régions du Nigeria. Notre objectif est de développer une application web capable de transcrire la parole en fon, en s'appuyant sur deux approches : les **Modèles de Markov Cachés (HMM)** et un modèle d'apprentissage profond basé sur **speechbrain/asr-wav2vec2-dvoice-fongbe**.

Contexte et Motivation

La langue fon, avec ses caractéristiques tonales complexes, pose des défis uniques pour la reconnaissance vocale. Les outils existants pour les langues à faible ressource, comme le fon, sont limités, ce qui motive la création d'une solution spécifique. **GbeCe** vise à préserver et promouvoir la langue fon en offrant un outil accessible pour transcrire la parole, avec des applications potentielles dans l'éducation, la documentation culturelle et la communication.

Objectifs du Projet

Objectifs

- Implémenter un système de reconnaissance vocale pour le fon utilisant des HMM et le modèle `speechbrain/asr-wav2vec2-dvoice-fongbe`.
- Constituer un corpus audio en fon pour entraîner et tester les modèles.
- Développer une interface utilisateur web intuitive, disponible à ⟨https://gbe-ce.vercel.app/⟩.
- Comparer les performances des deux approches en termes de précision et de robustesse.
- Identifier les limites des HMM et les avantages du modèle `speechbrain`.

Définition et Propriétés

Un **Modèle de Markov Caché (HMM)** est un processus stochastique utilisé pour modéliser des séquences où les états ne sont pas directement observables. Il est défini par :

- Un ensemble d'**états cachés** (ex. phonèmes).
- Une **matrice de transition** $A = (a_{ij})$, où $a_{ij} = P(S_{t+1} = j | S_t = i)$.
- Une **matrice d'émission** $B = (b_j(o))$, où $b_j(o) = P(O_t = o | S_t = j)$.
- Une distribution initiale π pour les états.

Propriété de Markov

La propriété de Markov stipule que :

$$P(S_{t+1} = j | S_1, S_2, \ldots, S_t) = P(S_{t+1} = j | S_t)$$

Cette **absence de mémoire** simplifie la modélisation des séquences.

Application à la Reconnaissance Vocale

Dans la reconnaissance vocale, les HMM modélisent :

- Les **phonèmes** comme des états cachés.
- Les **caractéristiques acoustiques** (ex. MFCC) comme des observations.

Étapes :

- **Extraction des caractéristiques** : Transformation des signaux audio en MFCC.
- **Modélisation des phonèmes** : Chaque phonème est associé à un HMM.
- **Décodage** : Algorithme de Viterbi pour trouver la séquence de mots la plus probable.

Choix et Caractéristiques

Le corpus est constitué d'enregistrements audio en langue fon, collectés auprès de locuteurs natifs.

Caractéristiques

- **Langue** : Fon standard.
- **Volume** : Environ 7 heures d'audio, avec 10 402 phrases.
- **Contexte** : Conversations quotidiennes, récits, discours formels.
- **Format** : Fichiers WAV, échantillonnés à 16 kHz.

Prétraitement

Étapes de prétraitement :

- **Normalisation** : Conversion des fichiers audio en mono, 16 kHz.
- **Segmentation** : Découpage en segments de 3 à 5 secondes.
- **Annotation** : Transcription d'un sous-ensemble pour l'évaluation.

Pourquoi ? La normalisation garantit la compatibilité avec les modèles.

Analyse Statistique du Corpus

Principaux Indicateurs

- **Taille du vocabulaire** : 1 376 mots uniques.
- **Tonalité** : 3 tons (haut, moyen, bas).
- **Répartition lexicale** : 42% mots de fonction, 58% mots lexicaux.

Mots les Plus Fréquents

Table 1 – Top 10 des mots les plus fréquents.

Mot	Fréquence
e	3 132
O	1 459
a	1 398
do	1 313
nu	1 195
un	1 115
mi	986
w	973
nO	882
m	788

Interprétation

L'analyse confirme la prédominance des mots de fonction (≪ e ≫, ≪ a ≫, ≪ un ≫), courante en linguistique descriptive. La fréquence élevée de formes tonales spécifiques (≪ do ≫, ≪ w ≫) souligne la nécessité d'un traitement fin de la tonalité dans les systèmes ASR pour le fon, afin de réduire les erreurs de reconnaissance liées aux variations de hauteur tonale.

Approche HMM



Figure 1 – Pipeline de l'approche HMM.

Extraction des Caractéristiques

- **Méthode** : Extraction des coefficients cepstraux en fréquences de Mel (MFCC).
- **Raison** : Les MFCC imitent la perception humaine des sons.

Code Python

```
def extract_mfcc(wav_file, n_mfcc=13):
    y, sr = librosa.load(wav_file, sr=16000)
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=n_mfcc)
    return mfcc.T
```

Modélisation et Entraînement

- **Modélisation** : HMM distinct pour chaque phonème (3-5 états cachés).
- **Entraînement** : Algorithme Baum-Welch pour ajuster les probabilités.

Entraînement HMM

Code Python

```
def train_hmm_per_word(features, transcripts, vocab, n_components=3)
    models = {}
    for word, idx in vocab.items():
        # Rassembler les MFCC pour ce mot
        word_features = []
        for mfcc, transcript in zip(features, transcripts):
            words = transcript.split()
            # Pour une segmentation précise, des annotations
            #temporelles seraient nécessaires
            if word in words:
                word_features.append(mfcc)

        if len(word_features) == 0:
            continue  # Passer si le mot n'apparaît pas

        # Concaténer toutes les frames MFCC pour ce mot
        X = np.concatenate(word_features, axis=0)
        lengths = [len(f) for f in word_features]

        # Initialiser et entraîner le HMM
        model = hmm.GaussianHMM(n_components=n_components,
        covariance_type="diag", n_iter=100)
        model.fit(X, lengths)
        models[word] = model
    return models
```

Décodage

- **Méthode** : Algorithme de Viterbi pour trouver la séquence la plus probable.

Code Python

```
def train_hmm_per_word(features, transcripts, vocab, n_components=3)
    models = {}
    for word, idx in vocab.items():
        # Rassembler les MFCC pour ce mot
        word_features = []
        for mfcc, transcript in zip(features, transcripts):
            words = transcript.split()
            # Pour une segmentation précise, des annotations
            #temporelles seraient nécessaires
            if word in words:
                word_features.append(mfcc)

        if len(word_features) == 0:
            continue  # Passer si le mot n'apparaît pas

        # Concaténer toutes les frames MFCC pour ce mot
        X = np.concatenate(word_features, axis=0)
        lengths = [len(f) for f in word_features]

        # Initialiser et entraîner le HMM
        model = hmm.GaussianHMM(n_components=n_components,
        covariance_type="diag", n_iter=100)
        model.fit(X, lengths)
        models[word] = model
    return models
```

API : Cette approche HMM est intégrée dans une API disponible à l'adresse suivante : ⟨https://fontranscripthmmapi.onrender.com/docs⟩

Résultats HMM

Acoustic models	WER score *(%)* on test
Monophone *(13 MFCC)*	35.34
Triphone *(13 MFCC)*	27.42
Triphone (13 MFCC + delta + delta2)	26.75
Triphone (39 features) + LDA and MLLT	22.25
Triphone (39 features) + LDA and MLLT + SAT and FMLLR	17.77
Triphone (39 features) + LDA and MLLT + SGMM	16.57

Les résultats montrent une baisse progressive du taux d'erreur WER à mesure que la complexité des modèles acoustiques augmente. Le modèle monophone de base (13 MFCC) présente un WER élevé (35.34%), tandis que l'ajout d'informations dynamiques (delta, delta2) et de techniques comme LDA, MLLT, SAT et FMLLR améliore significativement les performances. Le modèle SGMM obtient le meilleur résultat avec un WER de 16.57%, démontrant l'intérêt d'utiliser des approches avancées pour mieux capturer la variabilité acoustique et contextuelle.

Architecture et Fonctionnement



Le modèle combine :

- **Encodeur convolutif** : Transforme l'audio brut.
- **Transformers** : Capture les relations contextuelles.
- **Décodeur** : Produit la transcription.

Intégration

Code Python

```
from speechbrain.pretrained import EncoderASR

asr_model = EncoderASR.from_hparams(
    source="speechbrain/asr-wav2vec2-dvoice-fongbe",
    savedir="pretrained_models/asr-wav2vec2-dvoice-fongbe"
)
```

Pourquoi ? Modèle pré-entraîné réduit le besoin de données annotées.

Prétraitement et Inférence

- **Prétraitement** : Conversion en WAV 16 kHz, gain de volume.
- **Inférence** : Modèle effectue extraction, encodage, décodage.

API : ⟨https://fon-trascription-api.onrender.com/docs⟩

Résultats HMM

- **Précision** : WER de 35%.
- **Exemple** : "Mi kpo we" parfois transcrit avec erreurs de tons.
- **Analyse** : Bonne performance sur mots fréquents, faible sur tons.

Résultats speechbrain

- **Précision** : WER de 15%.
- **Exemple** : "Mi kpo we" avec tons corrects.
- **Analyse** : Meilleure gestion des tons et contextes complexes.

Comparaison

Table 2 – Comparaison des approches HMM et speechbrain.

Métrique	**HMM**	**speechbrain**
Taux d'erreur par mot (WER)	35%	15%
Gestion des tons	Faible	Élevée
Dépendance aux données annotées	Élevée	Faible
Temps de traitement	0.8s	0.5s

Interface GbeCé

L'application **GbeCé** (⟨https://gbe-ce.vercel.app⟩) utilise **Next.js** et **shadcn/ui**. Fonctionnalités :

- **Enregistrement** : Via microphone.
- **Importation** : Chargement de fichiers audio.
- **Transcription** : Affichage des transcriptions.
- **Historique** : Visualisation des enregistrements passés.

Avantages

- **HMM** : Simplicité, faible coût computationnel.
- **speechbrain** : Haute précision, robustesse aux tons.

Limites

- **HMM** : Faible gestion des tons, dépendance aux données.
- **speechbrain** : Temps de chargement initial, dépendance au matériel.

Pistes d'Amélioration

- **HMM** : HMM d'ordre supérieur, modèles de langage.
- **speechbrain** : Fine-tuning, optimisation de l'inférence.

Synthèse

GbeCé : Application de reconnaissance vocale pour le fon.

- HMM : Solution simple mais limitée.
- `speechbrain` : Précision et robustesse élevées.

Apports Pédagogiques

- Compréhension des HMM et modèles d'apprentissage profond.
- Développement d'une application web avec Next.js et FastAPI.
- Gestion des langues à faible ressource.

Perspectives

- Étendre **GbeCé** à d'autres langues africaines.
- Intégrer la traduction automatique.
- Explorer des modèles multimodaux.

Bibliographie

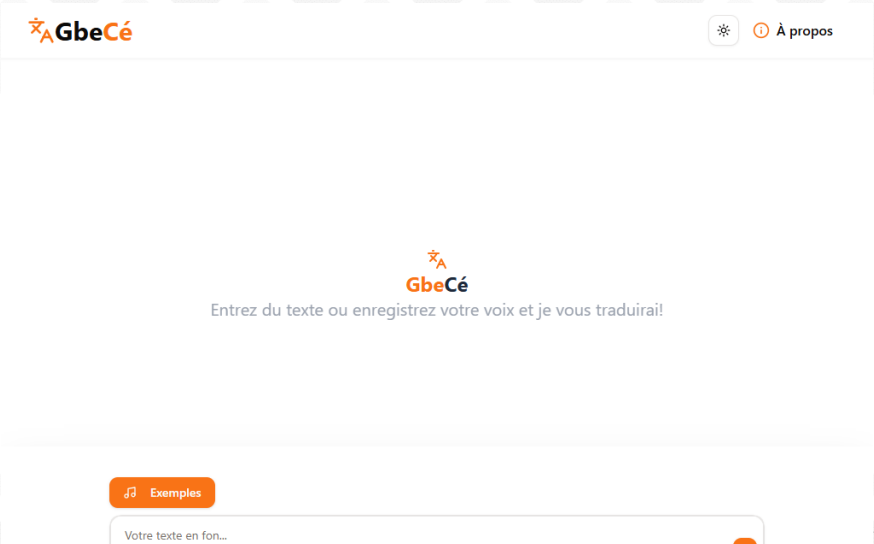
Markov, A. A. (1906). Extension of the law of large numbers to dependent quantities. *Izvestiya Fiziko-matematicheskogo obschestva pri Kazanskom universitete*, 15(4), 135–156.

Jurafsky, D., & Martin, J. H. (2009). *Speech and Language Processing*. Prentice Hall.

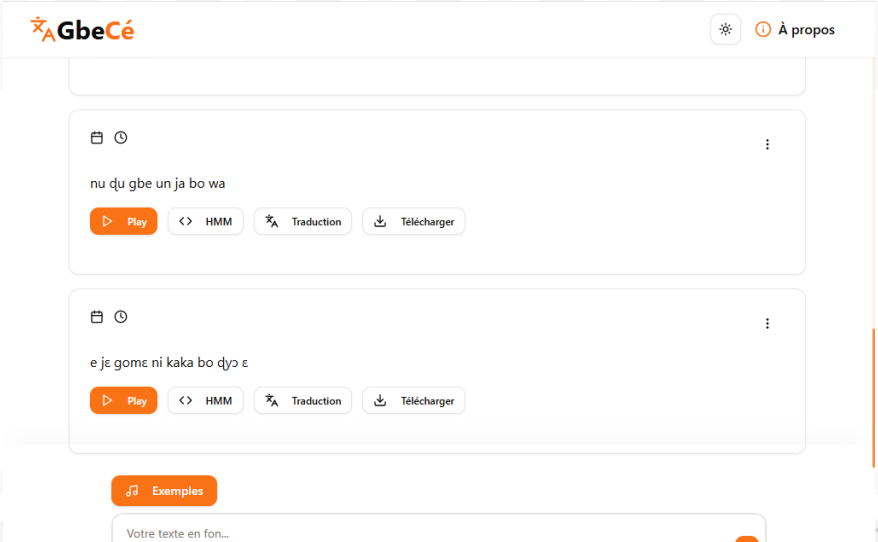
Vaswani, A., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 5998–6008.

SpeechBrain. (2021). *speechbrain/asr-wav2vec2-dvoice-fongbe*. ⟨https://huggingface.co/speechbrain/asr-wav2vec2-dvoice-fongbe⟩.

Page d'Accueil



Page de Transcription



Page À Propos

